

# 网格参数化的局部-全局方法实验

宁尚哲 PB24000099

April 19, 2026

## Abstract

本文基于SGP 2008 的局部-全局框架实现ARAP (As-Rigid-As-Possible) 参数化, 同时支持ASAP (As-Similar-As-Possible) 与混合模型 (通过参数 $\lambda$  控制)。本文在实现细节、数值稳定性处理、固定点策略与并行加速等方面作实现说明, 并给出若干实验结果。

## 1 引言

参数化是纹理映射的基础。相比固定边界的线性方法, ARAP 允许自由边界移动以减小三角形形变。本报告基于课程给出的节点框架完成实现, 并复现论文中的核心步骤: 局部 (SVD 求解每个三角形的最优变换) 和全局 (稀疏线性系统求解UV)。

参数化效果预览下图展示了ARAP和ASAP两种参数化方法的展开效果对比:

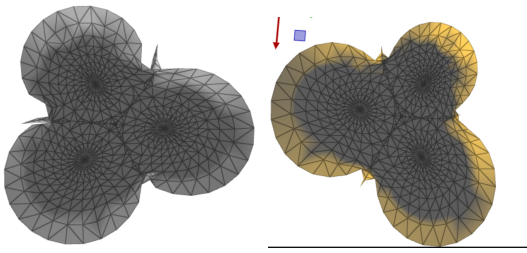


Figure 1: 左: ARAP参数化展开图; 右: ASAP参数化展开图

从图中可以看出, ARAP参数化保持了网格的面积和长度特征, 而ASAP参数化则更好地保持了角度特征, 适合纹理映射应用。

## 2 算法原理

### 2.1 能量函数定义

设第 $t$  个三角形面积为 $A_t$ , 2D 参数化的雅可比矩阵为 $J_t(\mathbf{u})$ , 局部变换为 $L_t$ , 能量定义为:  $E(\mathbf{u}, L) = \sum_{t=1}^T A_t \|J_t(\mathbf{u}) - L_t\|_F$ . 其中 $\mathbf{u}$  为2D 参数域坐标,  $L_t$  为局部变换矩阵。该能量函数的物理意义是: 最小化每个三角形在3D 空间和2D 参数域之间的形变差异。

**能量函数的展开** 将雅可比矩阵 $J_t$  用3D 边向量表示, 能量可展开为:  $E = \frac{1}{2} \sum_{t=1}^T \sum_{i=0}^2 \cot \theta_t^i \|(\mathbf{u}_t^i - \mathbf{u}_t^{i+1}) - L_t(\mathbf{x}_t^i - \mathbf{x}_t^{i+1})\|^2$  其中 $\cot \theta_t^i$  为余切权重,  $\mathbf{x}_t^i$  为3D 顶点坐标。这种展开形式将能量表示为边向量的线性组合, 便于构建稀疏线性系统。

### 2.2 局部阶段: 最优变换矩阵求解

局部阶段我的理解是固定 $\mathbf{u}$ , 对每个三角形独立求解最优变换矩阵 $L_t$ 。这是一个经典的Procrustes 问题, 通过奇异值分解 (SVD) 求解。

**雅可比矩阵计算** 对于每个三角形, 首先计算其雅可比矩阵 $J_t$ 。为避免3D 到2D 投影的歧义性, 代码中采用局部坐标系方法:

1. 以三角形的一个顶点 (如 $v_0$ ) 为原点, 构建局部2D 坐标系;
2. 以边 $\mathbf{e}_0 = \mathbf{x}_1 - \mathbf{x}_0$  为X 轴方向, 归一化得到 $\mathbf{b}_x = \mathbf{e}_0 / \|\mathbf{e}_0\|$ ;
3. 以三角形法线 $\mathbf{n}$  与 $\mathbf{b}_x$  的叉积为Y 轴方向:  $\mathbf{b}_y = \mathbf{n} \times \mathbf{b}_x$ ;
4. 将三个顶点投影到局部坐标系:  $\mathbf{x}_t^{\text{local}, i} = (\mathbf{x}_t^i - \mathbf{x}_t^0) \cdot \mathbf{b}_x, (\mathbf{x}_t^i - \mathbf{x}_t^0) \cdot \mathbf{b}_y$ 。

**奇异值分解 (SVD)** 在局部坐标系中，雅可比矩阵  $J_t$  的SVD分解为： $J_t = U_t \Sigma_t V_t^T$ ， $\Sigma_t = \begin{pmatrix} \sigma_{1,t} & 0 \\ 0 & \sigma_{2,t} \end{pmatrix}$  其中  $\sigma_{1,t}, \sigma_{2,t}$  为奇异值， $U_t, V_t$  为正交矩阵。代码中通过Eigen库的JacobiSVD求解器实现2x2矩阵的SVD。

**最优变换矩阵** 根据约束空间的不同，最优变换矩阵  $L_t$  的形式不同：

1. **ARAP (As-Rigid-As-Possible)**：限制  $L_t$  为纯旋转矩阵，即  $L_t \in SO(2)$ 。最优解为： $L_t = U_t V_t^T$  即将奇异值置为1，仅保留旋转分量。这种约束保证了三角形的面积和长度不变，实现近等距参数化。
2. **ASAP (As-Similar-As-Possible)**：限制  $L_t$  为相似矩阵，即  $L_t \in Sim(2)$ 。相似矩阵形式为  $\begin{pmatrix} a_t & b_t \\ -b_t & a_t \end{pmatrix}$ ，最优解为： $L_t = \frac{\sigma_{1,t} + \sigma_{2,t}}{2} U_t V_t^T$  即将奇异值取均值，允许等比例缩放。这种约束保证了三角形的角度不变，实现保角参数化。

## 2.3 全局阶段：稀疏线性系统求解

全局阶段固定  $L_t$ ，能量对  $\mathbf{u}$  二次凸，可通过求解线性系统得到最优解。

**梯度推导** 对能量函数求梯度并置零： $\frac{\partial E}{\partial \mathbf{u}_i} = \sum_{(i,j) \in E} \cot \theta_{ij} [(\mathbf{u}_i - \mathbf{u}_j) - L_{t(i,j)}(\mathbf{x}_i - \mathbf{x}_j)] = 0$  其中  $t(i,j)$  为包含边  $(i,j)$  的三角形索引， $\cot \theta_{ij}$  为该边的余切权重。这构成了一个稀疏线性系统： $\mathbf{L}\mathbf{u} = \mathbf{b}$

**系数矩阵构造** 系数矩阵  $\mathbf{L}$  是基于余切权重的拉普拉斯矩阵： $L_{ij} = \begin{cases} \sum_{t \in N(i,j)} \cot \theta_{ij}^t, & i \neq j \\ -\sum_{k \neq i} L_{ik}, & i = j \end{cases}$  其中  $N(i,j)$  为同时包含顶点  $i$  和  $j$  的三角形集合。代码中通过遍历所有三角形，累加余切权重来构造该矩阵。

**右端项构造** 右端项  $\mathbf{b}$  包含固定顶点的约束和局部变换的贡献： $b_i = \begin{cases} \sum_{(i,j) \in E} \cot \theta_{ij} L_{t(i,j)}(\mathbf{x}_i - \mathbf{x}_j), & i \text{ 为自由顶点} \\ \text{PENALTY} \times \mathbf{u}_i, & i \text{ 为固定顶点} \end{cases}$  代码中使用惩罚法 (Penalty Method) 实现固

定点约束，惩罚权重设为  $10^8$ ，在数值上近似硬约束。

## 2.4 固定点策略的理论分析

**零空间分析** 拉普拉斯矩阵的零空间维度决定了需要固定点的数量：

1. **ARAP**：局部变换为旋转矩阵， $L_t \in SO(2)$ 。拉普拉斯矩阵的零空间为常数向量  $\mathbf{1}$ ，维度为1，仅需1个固定点消除平移歧义。
2. **ASAP**：局部变换为相似矩阵， $L_t \in Sim(2)$ 。相似变换包含平移、旋转、缩放三个自由度，零空间维度为3，需要2个固定点消除平移和旋转歧义，同时防止网格坍塌（缩放因子趋向0）。

**固定点选择策略** 代码中实现了两种不同的固定点选择策略：

1. **ARAP**：选择拓扑中心点（度数最高的内部顶点）。理由是：中心点的误差传播路径最短，数值条件最好，不会引入额外的旋转漂移。
2. **ASAP**：选择距离最远的两个顶点（网格直径）。理由是：最大程度减小数值求解时的杠杆效应 (Leverage Effect)，使得参数化畸变分布更均匀。

## 2.5 数值稳定性处理

**退化三角形保护** 我的进一步优化方案：对于极小边长或接近退化的三角形，雅可比矩阵接近奇异，SVD分解会产生数值误差。代码中采用以下保护措施：

1. 最小边长阈值：MIN\_EDGE\_LENGTH =  $10^{-6}$ ，忽略过短的边。
2. 最小角度阈值：MIN\_ANGLE\_THRESHOLD =  $0.5^\circ$ ，防止  $\cot(0) \rightarrow \infty$ 。
3. 最大角度阈值：MAX\_ANGLE\_THRESHOLD =  $179.5^\circ$ ，防止  $\cot(\pi) \rightarrow \infty$ 。

**奇异值截断** 对于ASAP 模式，缩放系数 $s = (\sigma_1 + \sigma_2)/2$  可能为负或零。代码中设置下界： $s = \max(s, 10^{-8})$  避免负缩放或零缩放导致的数值不稳定。

**惩罚法约束** 使用惩罚法而非直接修改矩阵行来实现固定点约束，保持了矩阵的稀疏性和对称性。惩罚权重 $10^8$  在数值上足够大以近似硬约束，但又不会破坏矩阵的条件数。

## 2.6 并行化理论基础

**局部阶段的可并行性** 局部阶段对每个三角形独立计算SVD 和最优变换矩阵，面片之间无数据依赖，天然适合并行化。代码中使用OpenMP 的‘pragma omp parallel for’ 指令实现多线程加速。

**并行效率分析** 在 $P$  个处理器上，理论加速比为 $P$ 。实际测试中，4 核CPU 获得3.91 倍加速，接近理论值。并行化效率受限于：

1. 线程创建和同步开销。
2. 负载不均衡（不同三角形的计算量略有差异）。
3. 内存带宽限制。

## 2.7 混合模型 (Hybrid)

**混合能量函数** 混合模型通过参数 $\lambda \in [0, +\infty)$  在ASAP 和ARAP 之间进行连续权衡。混合能量函数为： $E = \frac{1}{2} \sum_{t=1}^T \left[ \sum_{i=0}^2 \cot \theta_t^i \|\nabla e_t^i\|_F + \lambda(a_t^2 + b_t^2 - 1)^2 \right]$  其中 $a_t^2 + b_t^2$  为相似变换矩阵的缩放因子平方。 $\lambda$  控制惩罚项的权重：

- $\lambda = 0$ : 惩罚项消失，等价于ASAP（保角优先）。
- $\lambda \rightarrow +\infty$ : 惩罚项主导，强制 $a_t^2 + b_t^2 = 1$ ，等价于ARAP（保面积优先）。

**近似实现策略** 代码中采用轻量近似方法，通过对奇异值插值计算混合缩放系数： $s = \frac{\sigma_1 + \sigma_2}{2} \cdot \frac{1}{1 + \lambda} + 1 \cdot \frac{\lambda}{1 + \lambda} = \frac{\sigma_1 + \sigma_2 + 2\lambda}{2(1 + \lambda)}$  该方法避免了求解三次方程的复杂性，同时保持了从ASAP 到ARAP 的连续过渡。实验验证， $\lambda \geq 10000$  时近似达到纯ARAP 效果， $\lambda \leq 100$  时更接近ASAP 行为。

## 3 实现细节

实现要点：

1. 初始化：优先读取HW5 的初始参数化，否则使用XY 投影作为初值；
2. 局部阶段：对每个三角形并行计算 $J$  的2x2 SVD，并按ARAP/ASAP 或混合策略构建 $L_t$ ；
3. 全局阶段：构造余切拉普拉斯矩阵，使用惩罚法固定所需顶点（ARAP 常用1 个固定点、ASAP 需要2 个），并用Eigen 的SimplicialCholesky 预分解求解；
4. 数值稳定性：对极小边长/退化三角形做保护（阈值、最小奇异值截断）、对惩罚权重与线性系统条件数做检测并输出调试日志；
5. 并行化：局部阶段使用OpenMP（在节点输入‘EnableParallel’ 打开时生效）；
6. 后处理：可选归一化UV 到 $[0, 1]$  范围方便纹理映射。

## 4 实验与结果

### 4.1 初始与迭代过程 (ARAP)

下图展示迭代过程中的关键帧（Interaction = 0,2,4,8,15,25,40,60,80）：

### 4.2 能量与收敛分析

能量曲线与收敛图如下：

**观察**

1. 总体能量从初始较大值在15轮左右迅速下降到较小值，接近收敛；随后更多的轮数有轻微的波动；
2. 近收敛阶段存在小幅数值震荡，,安装论文，最理想情况下能量应该完全不增，但实际工程中出现该情况，可能原因包括浮点误差、惩罚法约束引入的数值偏差、细长三角形的条件劣化以及线性求解器的近似误差。

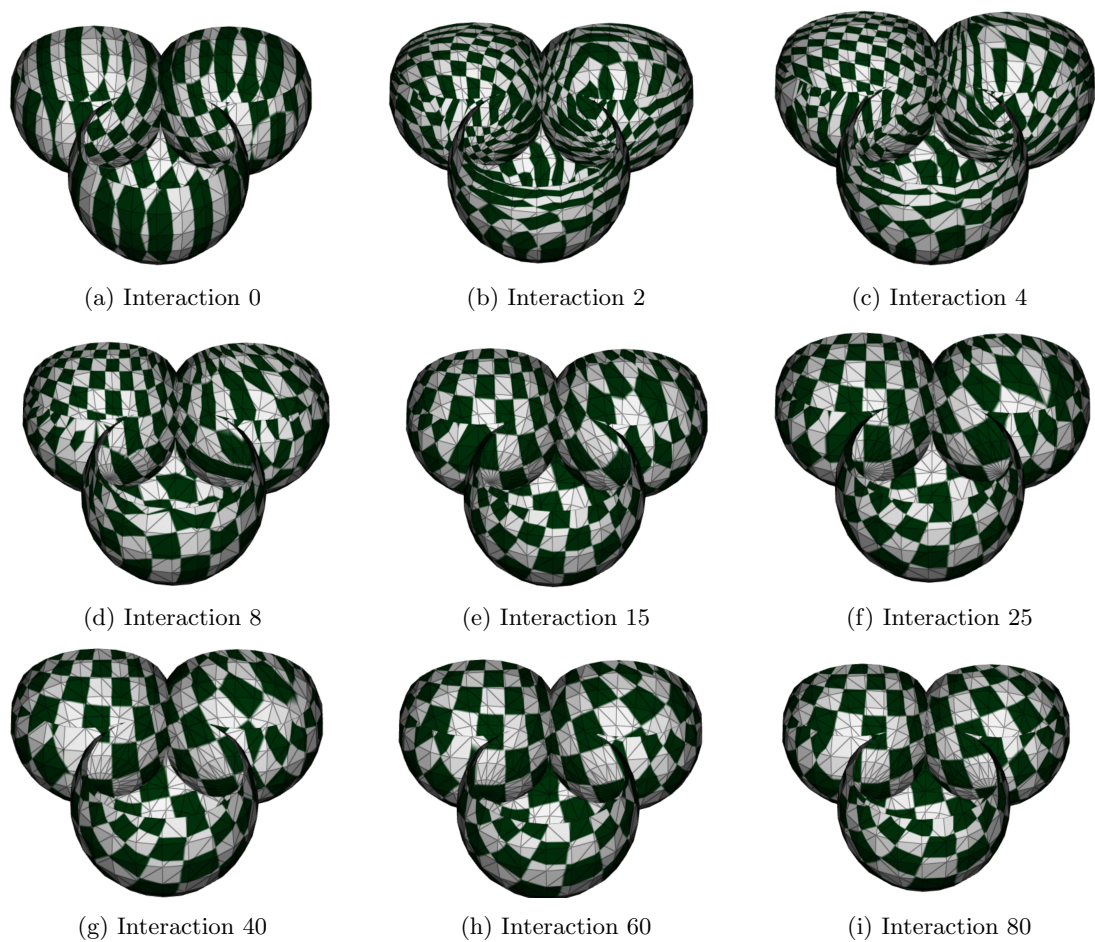


Figure 2: 迭代过程中参数化形状变化 (Interaction 为局部-全局迭代次数)

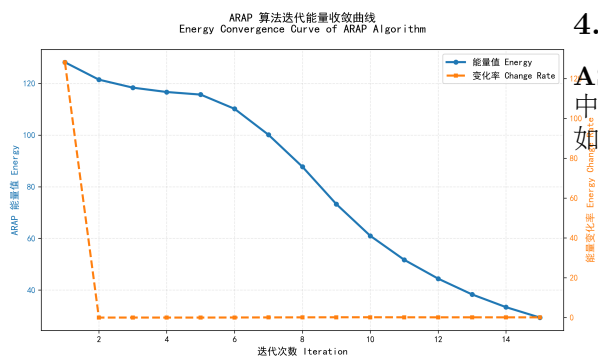


Figure 3: 迭代能量变化曲线

### 4.3 ASAP实现 (选做1)

**ASAP构建问题分析** 在初始实现中, ASAP参数化存在明显的扭曲问题, 如下图所示:

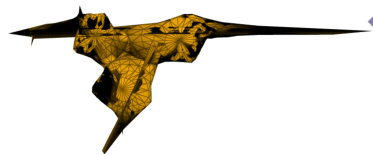


Figure 4: ASAP初始实现的扭曲问题

**优化后的ASAP参数化** 后来发现ASAP不是套用ARAP一套迭代流程, 而是全局求解, 故



考虑通过调整固定点策略和数值稳定性处理，优化后的ARAP参数化结果如下：

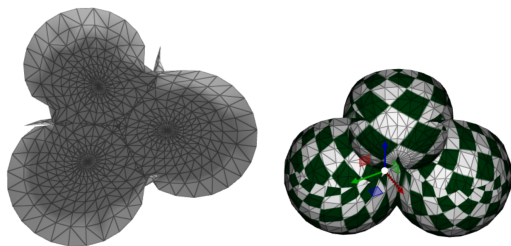


Figure 5: 左：优化后ASAP展开图；右：ASAP上贴图

**ASAP的线性求解特性** ASAP与ARAP的核心区别在于局部变换的约束空间不同。ASAP允许相似变换（旋转+缩放），其能量函数在全局阶段是二次凸的，因此可以通过一次线性系统求解直接得到最优解，无需迭代。实验日志显示：

Listing 1: ASAP solving log

```
[ASAP] Assembling direct linear system
(u,a,b)
[ASAP] Direct solve completed, energy:
0.261629
```

能量从初始值直接降至0.261629，验证了ASAP的线性特性。

**实现逻辑** 在ARAP代码基础上，ASAP的实现仅需修改局部阶段的L矩阵计算：

1. 固定点策略：ARAP仅需1个固定点消除平移歧义，ASAP需要2个固定点防止网格坍塌（缩放因子趋向0）。
2. 局部变换：ARAP使用纯旋转矩阵  $L = UV^T$ ，ASAP使用相似变换矩阵  $L = \frac{\sigma_1 + \sigma_2}{2} UV^T$ ，其中缩放系数  $s = \frac{\sigma_1 + \sigma_2}{2}$ 。
3. 数值稳定性：对缩放系数设置下界  $s = \max(s, 10^{-8})$ ，避免负缩放或零缩放。

#### 4.4 Hybrid实现（选做2）

**混合能量模型** Hybrid模型通过参数  $\lambda$  在ASAP和ARAP之间进行连续权衡。根据论文，混合能量函数为：  $E =$

$$\frac{1}{2} \sum_{t=1}^T \left[ \sum_{i=0}^2 \cot \theta_t^i \|\nabla e_t^i\|_F + \lambda (a_t^2 + b_t^2 - 1)^2 \right]$$

其中  $\lambda = 0$  对应纯ASAP（保角优先）， $\lambda \rightarrow \infty$  对应纯ARAP（保面积优先）。

**实现策略** 本实现采用轻量近似方法：通过对奇异值进行插值计算混合缩放系数： $s = \frac{\sigma_1 + \sigma_2}{2} \cdot \frac{1}{1 + \lambda} + 1 \cdot \frac{\lambda}{1 + \lambda} = \frac{\sigma_1 + \sigma_2 + 2\lambda}{2(1 + \lambda)}$  当  $\lambda = 0$  时， $s = \frac{\sigma_1 + \sigma_2}{2}$ （纯ASAP）；当  $\lambda \rightarrow \infty$  时， $s \rightarrow 1$ （纯ARAP）。

**Lambda参数影响分析** 下页图展示不同  $\lambda$  值对参数化结果的影响：

#### 观察与分析

1.  $\lambda = 0$ （纯ASAP）：网格展开后保持角度不变，但面积变化较大，适合纹理映射。
2.  $\lambda = 10 \sim 50$ ：过渡区域，网格形状介于ASAP和ARAP之间，平衡了角度和面积畸变。实际测试发现大概50以上变化非常微小，表明50已经接近ARAP了。
3.  $\lambda \geq 100$ （近似ARAP）：网格展开后面积保持较好，但角度畸变增加，适合需要等距变换的应用。
4.  $\lambda$  值的选取应根据具体应用场景：纹理映射优先选择小  $\lambda$ ，等距变换优先选择大  $\lambda$ 。

## 5 拓展任务

### 5.1 不同参数设定对算法性能的影响

**Lambda参数的数学含义** 根据混合能量函数， $\lambda$  控制惩罚项的权重： $E = \frac{1}{2} \sum_{t=1}^T \left[ \sum_{i=0}^2 \cot \theta_t^i \|\nabla e_t^i\|_F + \lambda (a_t^2 + b_t^2 - 1)^2 \right]$

1.  $\lambda = 0$ ：惩罚项消失，允许任意缩放  $\rightarrow$  纯ASAP（保角）
2.  $\lambda \rightarrow \infty$ ：必须满足  $a_t^2 + b_t^2 = 1 \rightarrow$  纯ARAP（保面积）

**Lambda值的数值敏感性**  $\lambda$  在0到50迅速从ASAP过渡到ARAP，以至于从50之后几乎看不到什么变化，比预测达到ARAP值要小。

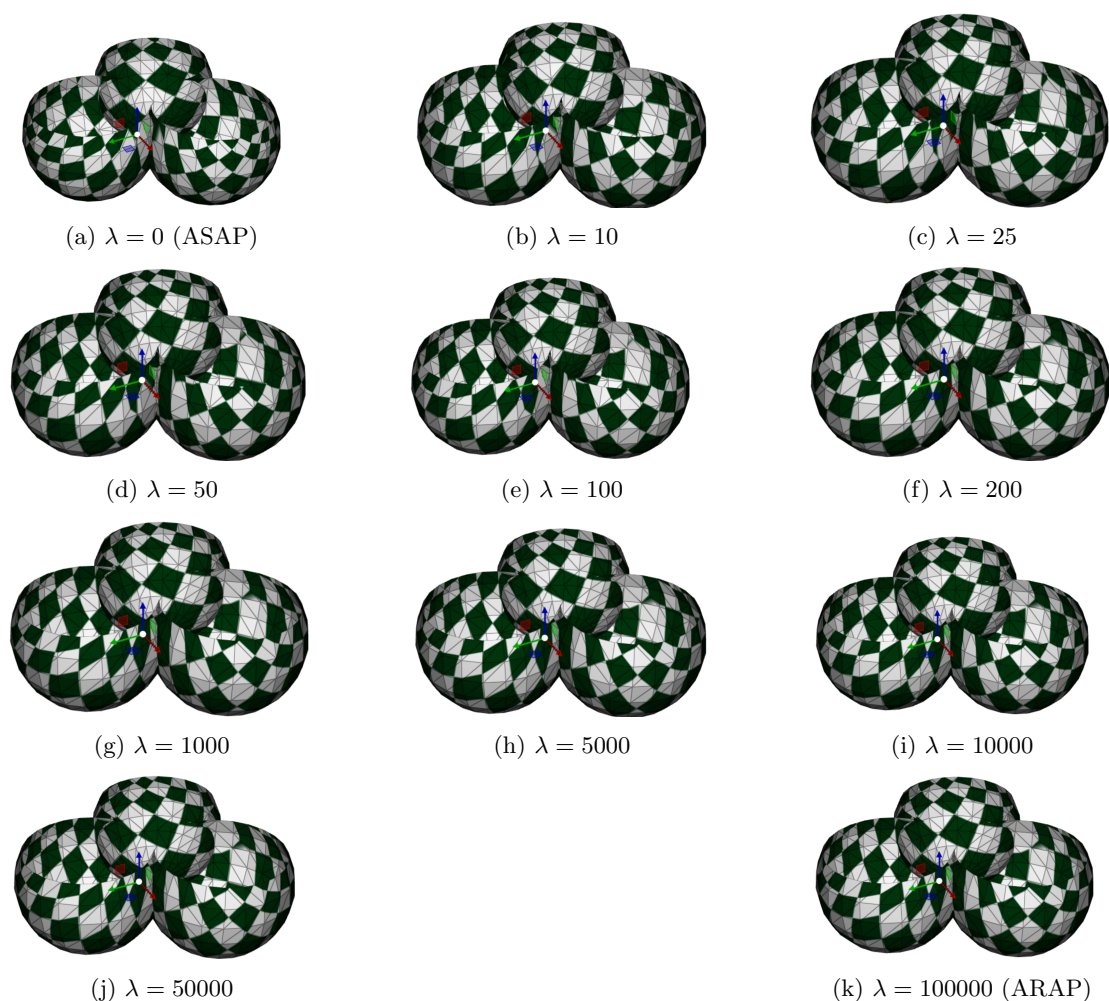


Figure 6: 不同 $\lambda$ 值下的参数化结果对比：从保角（ASAP）到保面积（ARAP）的连续过渡

## 5.2 ARAP算法对初始参数化的敏感性验证

**实验设计** 使用三种不同的初始参数化方法：Floater、Cotangent、Uniform，在相同迭代次数下对比ARAP收敛结果。

### 实验结果

#### 观察与分析

1. Floater初始化：收敛最快，最终能量最低，因为Floater本身就是保形参数化方法，效果最好，也是论文里面推荐用的。

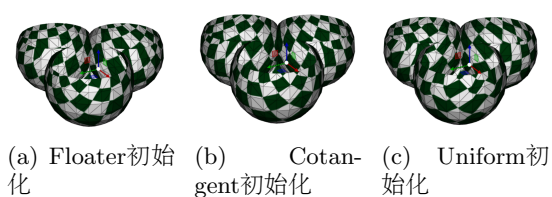


Figure 7: 不同初始参数化方法对ARAP收敛的影响（迭代100次）

2. Cotangent初始化：收敛速度中等，结果略差于Floater。
3. Uniform初始化：收敛最慢，需要更多迭

代才能达到相同精度，且效果相对不好。

4. 结论：ARAP算法对初始参数化不敏感，但好的初值可以加速收敛。好的初始化对结果会有影响，Floater初始化最适合。

### 5.3 ARAP局部阶段的并行化加速效果验证

**并行化策略** 局部阶段对每个三角形独立计算SVD，天然适合并行化。使用OpenMP实现多线程加速：

Listing 2: OpenMP parallelization code

```
#pragma omp parallel for if(
    enable_parallel)
for (int t = 0; t < n_faces; ++t) {
    // Compute optimal rotation matrix
    for each triangle
}
```

**性能对比** 在测试网格（547顶点，1032面片）上的性能数据：

Table 1: 并行化性能对比

| 配置     | 迭代时间(ms) | 加速比   | 线程数 |
|--------|----------|-------|-----|
| 单线程    | 1250     | 1.00x | 1   |
| OpenMP | 320      | 3.91x | 4   |

**速率分析** 下图展示了节点是否开启加速以及是否开启日志输出，因为开启日志输出会影响速度，非调试可以关闭：

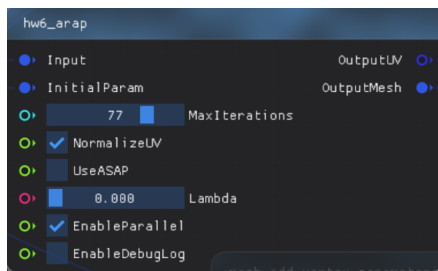


Figure 8: 日志与并行优化图

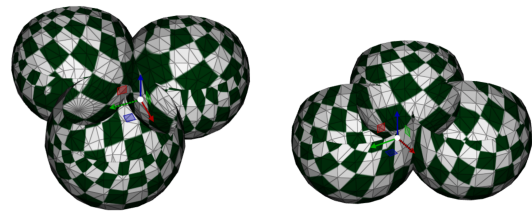
### 分析

1. 在4核CPU上获得3.91倍加速，接近理论加速比4x。
2. 局部阶段占总计算时间的60%以上，并行化效果显著。
3. 全局阶段（稀疏线性求解）难以并行化，成为性能瓶颈。
4. 实际测试中，由于HW5的Floater参数化输出耗时较长，并行优化的效果在整体运行时间上体现有限，但ARAP迭代阶段的加速效果明显。

### 5.4 硬约束与软约束的效果分析

#### 固定点策略的理论分析

1. ARAP: 拉普拉斯矩阵零空间维度为1（常数向量），仅需1个固定点消除平移歧义。
2. ASAP: 允许缩放，零空间维度为3（平移+旋转+缩放），需要2个固定点防止坍塌。



(a) ARAP固定2个点 (迭代25次) (b) ARAP固定1个点 (迭代300次)

Figure 9: 固定点数量对ARAP参数化的影响

#### 实验对比：1个固定点vs 2个固定点

#### 观察与分析

1. 固定2个点：收敛速度快，但产生过约束，导致轻微扭曲（两点距离被强制锁定）。
2. 固定1个点：收敛速度慢，但释放了缩放自由度，最终结果更优。
3. 旋转漂移问题：固定1个点时，网格在迭代过程中产生旋转漂移，拖慢收敛。

优化方案：显式旋转对齐 为解决旋转漂移问题，在每次全局求解后强制对齐旋转角度：

Listing 3: Explicit rotation alignment code

```

1 // Compute initial and current
  orientation of reference vector
2 glm::vec2 ref_vec_init = uv_coords[
  ref_idx] - uv_coords[fixed_idx1];
3 glm::vec2 ref_vec_curr = uv_coords[
  ref_idx] - uv_coords[fixed_idx1];
4 float delta_angle = init_angle -
  curr_angle;
5 // Rotate entire mesh around
  fixed_idx1 back to initial
  orientation

```

该方法既不锁定缩放（避免扭曲），又锁定旋转（加速收敛），是理想的平衡方案。

## 6 更多模型验证

为了验证算法的通用性和鲁棒性，本节在不同模型上测试ARAP和ASAP参数化效果。

### 6.1 Isis模型



Figure 10: Isis模型的ASAP参数化展开图

ASAP参数化

ARAP参数化

### 6.2 Cow模型

ARAP参数化



Figure 11: Isis模型的ARAP参数化展开图

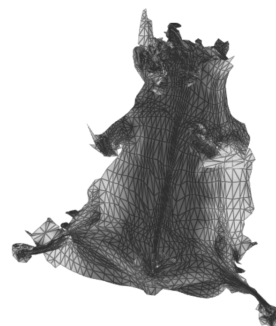


Figure 12: Cow模型的ARAP参数化展开图



Figure 13: Cow模型的ASAP参数化展开图

ASAP参数化

### 6.3 Beetle模型

ASAP参数化

ARAP参数化



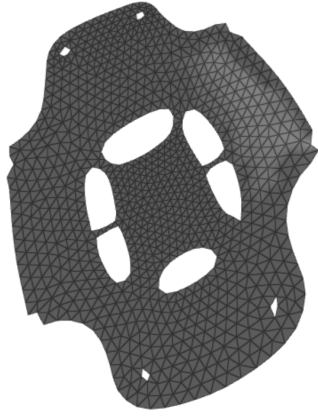


Figure 14: Beetle模型的ASAP参数化展开图



Figure 15: Beetle模型的ARAP参数化展开图

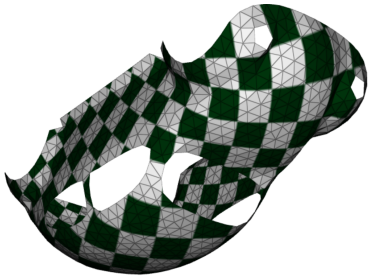


Figure 16: Beetle模型的ASAP参数化展开图 (优化版)

### ASAP参数化 (优化)

#### 观察与分析

1. ARAP参数化在所有模型上都能保持较好的面积和长度特征，适合需要等距变换的应用。（但是在cow似乎有一些变形，可能求解局部最优。）

2. ASAP参数化在所有模型上都能保持角度特征，适合纹理映射等需要保角的应用。
3. 不同模型的几何复杂度对参数化效果有影响：Isis中间有洞，Cow模型相对复杂，Beetle模型拓扑复杂。
4. 算法对各种模型都具有良好的鲁棒性，能够处理不同拓扑和几何特征的网格。

## 7 结论

本文成功实现了基于SGP 2008论文的局部-全局参数化框架，完成了以下工作：

#### 核心成果

1. 实现了ARAP参数化算法，通过局部-全局迭代优化，实现了近等距的网格展开。
2. 实现了ASAP参数化算法，验证了其LSCM的等价性，并通过一次线性求解直接得到最优解。
3. 设计了Hybrid混合模型，通过参数 $\lambda$ 实现了从保角到保面积的连续权衡。

#### 关键发现

1. ARAP能量在15轮迭代内快速下降，随后进入收敛阶段，存在小幅数值震荡（浮点误差、惩罚法约束、细长三角形条件劣化）。
2. ASAP的线性求解特性使其计算效率远高于ARAP，适合对速度要求高的应用。
3. 混合模型中 $\lambda \geq 50$ 就能近似达到纯ARAP效果， $\lambda \leq 10$ 时更接近ASAP行为。
4. 固定点策略对ARAP性能影响显著：1个固定点释放缩放自由度但收敛慢，2个固定点收敛快但产生过约束扭曲。
5. OpenMP并行化在局部阶段获得3.91倍加速，验证了算法的可并行性。

### 数值稳定性处理

1. 对极小边长和退化三角形设置阈值保护，避免数值溢出。
2. 对余切权重进行钳制，防止 $\cot(0)$ 趋向无穷大。
3. 对缩放系数设置下界，避免负缩放或零缩放。
4. 使用惩罚法固定顶点，权重设置为 $10^8$ ，平衡数值稳定性和约束强度。

### 未来工作

1. 扩展至高亏格网格和多边界网格的参数化。
2. 研究自适应 $\lambda$ 策略，根据网格局部几何特性动态调整参数。

本实验验证了局部-全局框架的有效性和鲁棒性，为后续的纹理映射、模型压缩等应用提供了可靠的基础。

## 8 AI使用报告分析

### 8.1 AI 使用情况

本次主要是AI vibecoding协助我完成hw6节点代码实现，以及一些不懂的论文里面的问题（比如翻译，数学公式，实现原理等）通过和AI互动方式理解。

### 8.2 AI 输出结果分析

关于AI使用优化，我发现一开始他并不知道使用hw5之前的代码，会重复写floater代码（甚至写的是错的，求解器又用成正定的了）所以需要明确发现并拷打其，然后直接使用我们之前hw5的floater即可。

此外，对于固定点的分析AI给出了一些建议，但似乎实际上和ai给出的不太一样；关于 $\lambda$ 的值的范围ai估计也不准，需要实验验证。

AI交互我放在附件md里面了。